Monetization & Multi-Salon User Access Integration Summary

Date: April 16, 2026
Status: Planning & Documentation Complete ✅

> Note: This document is an intermediate planning snapshot. The authoritative roadmap is now the expanded 20-week plan in `MULTITENANT_COMPANION_EXECUTION_BLUEPRINT.md` and prompt packs `MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md` through `MULTITENANT_WEEKS_13_TO_20_COPILOT_PROMPTS.md`.

Executive Summary

You've identified two major strategic features to add before development begins:

1. **Multi-Salon User Access**: End-users subscribe to multiple salons (nails, massage, hairdresser, etc.) via ONE Zarkili account with a unified dashboard showing all subscriptions and unread message badges per salon.
2. **Stripe Payment Integration**: Businesses (salon tenants) subscribe to the Zarkili platform on a monthly/annual basis with tiered features. Implementation happens after pilot launch (Week 13 onwards).

This summary has been superseded by the expanded 20-week roadmap where these features are fully integrated.

Your Specific Decisions (Captured)

Feature	Decision
Subscription Tiers	Multiple (Starter, Professional, Enterprise planned)
Billing Frequency	Monthly + Annual (annual with discount)
Feature Gating	Full gating based on subscription tier
Free Trial	14 days
User Model	ONE account, multiple subscriptions
Payment Failure	Grace period + suspension indicator
Payment Integration Timeline	Post-Week 12 (not required for pilot)

What's Been Updated (4 Core Documents)

1. MULTITENANT_STRATEGY_AND_FEATURE_AUDIT.md

Changes:

- Added "Multi-Salon User Access Model" to the "Add New (Not Present)" section
- Added "Subscription and Billing" (tiered, Stripe, gating, grace period) to the same section
- Expanded Phase 6 (Platform Operations and Monetization) with:
  - Multi-tier subscription model details (Starter/Professional/Enterprise)
  - Stripe integration specifics
  - Feature gating per tier
  - 14-day free trial
  - Payment failure handling (grace period + suspension)
  - Multi-Salon User Access dashboard features

Key Insight: Phase 6 is now explicitly about both **platform monetization** (how you get paid) AND **multi-salon UX** (how end-users access multiple salons).

---

2. MULTITENANT_COMPANION_EXECUTION_BLUEPRINT.md

This is the most significantly updated document. Changes span schema, indexes, and roadmap:

Schema (Section 1.2)

Extended `tenantUsers` Collection:

- Added `subscription` JSON object with fields:
  - `tier`: starter | professional | enterprise
  - `status`: trialing | active | past_due | suspended | cancelled
  - `billingCycle`: monthly | annual
  - `startDate`, `trialEndsAt`, `nextBillingDate`, `suspendedAt`, `suspensionReason`
- These fields are set up in Week 2 but NOT enforced until Stripe integration (Week 13+)

New `subscriptions` Collection (Platform-Level):

```
Purpose: Track payment/subscription lifecycle per tenant
Fields: subscriptionId, tenantId, customerId (Stripe), tier, status,
         billingCycle, dates, payment method, pricing, failureCount, gracePeriod
```

New `userTenantAccess` Collection (Performance Index):

```
Purpose: Fast lookups for "what tenants does this user have?" + unread counts
Fields: userId, tenantId, accessLevel, subscriptionStatus, subscribedAt,
         unreadMessageCount, lastMessageAt, lastAccessedAt
Note: Denormalized mirror of key fields from tenantUsers for efficient dashboard
queries
```

Indexes (Section 1.3)

Added 6 new indexes:

- `subscriptions`: tenantId + status
- `subscriptions`: status + nextBillingDate
- `subscriptions`: status + gracePeriodEndsAt (for payment failure management)
- `userTenantAccess`: userId + accessLevel
- `userTenantAccess`: userId + subscriptionStatus + lastAccessedAt (dashboard listing)
- `userTenantAccess`: tenantId + subscriptionStatus (tenant's active user count)

Roadmap (Section 2)

Added "Critical Architecture Note" explaining multi-salon phasing:

- Week 2: Extend tenantUsers + introduce userTenantAccess index
- Week 3-5: Build all core features (single-tenant context)
- **Week 5.5 (NEW)**: Multi-Salon Home Dashboard and Context Switcher
- Post-Week 12: expanded execution block (Weeks 13-20) for payments, onboarding, marketplace, and AI

Week 2 Updated:

- Now explicitly includes "subscribe with subscription metadata" and userTenantAccess index
- Quality gate checks that subscription fields are properly structured

Week 5.5 (NEW TASK):

```
Objectives:
- User home screen showing all subscribed salons
- Unread message badge counts per salon
- Context switching and deep-linking
- Marketplace discovery if no subscriptions

Deliverables:
- MultiSalonDashboardScreen component
- Unread aggregation service
- Context switcher navigation
- Salon card UI with logo, name, next appointment, badges, quick actions
- Loyalty points per salon

Quality Gates:
- Unread counts match individual salons
- Context switch preserves session
- Deep links work from push notifications
- Empty/error states handled
```

Post-Week 12: Weeks 13-20 (Expansion Timeline):

```
Week 13 - Payment Processing Integration:
- Stripe API integration
- Monthly/annual billing setup
```

```
- WebHook handlers for payment events
- Invoice generation

Week 14 - Feature Gating & Subscription Lifecycle:
- Feature flag service by tier
- Subscription state transitions (active→past_due→suspended)
- Grace period management
- Free trial countdown UI

Weeks 15-16 - Onboarding Expansion:
- Salon onboarding wizard implementation
- Client onboarding integration (guest + full account upgrade)

Weeks 17-18 - Marketplace Expansion:
- Marketplace core launch and anti-client-theft constraints
- Marketplace analytics and moderation controls

Weeks 19-20 - AI Expansion:
- AI chat, scheduling, marketing, and retention assistants
- AI risk scoring and marketplace personalization
```

Summary Timeline Added:

- Weeks 1-12: Core platform (Zara pilot ready)
- Week 5.5: Multi-salon dashboard (in parallel with booking)
- Weeks 13-20: Payment, onboarding, marketplace, and AI expansion

---

3. MULTITENANT_WEEKS_1_TO_4_COPILOT_PROMPTS.md

Changes to Week 2 Tasks:

Task 2.2 Updated (Tenant User Role Mapping):

- Now includes subscription metadata fields in prompt requirements
- Validates subscription state shape
- Includes guards that validate future Stripe integration compatibility
- Note: Does NOT enforce feature gating yet (that's Week 14 work)

NEW Task 2.2.5 (User Tenant Access Index):

- Implements userTenantAccess collection
- Repository methods: createUserTenantAccess, updateUnreadMessageCount, getUserTenants, getTenantUsers, deactivateUserTenantAccess
- Critical: Includes tests for synchronization between tenantUsers and userTenantAccess
- Requires documentation of all sync points for future developers

Why These Tasks Matter:

- Both are in Week 2 (foundation phase)
- They set up the data structure and queries needed for multi-salon dashboard in Week 5.5

- They pre-stage subscription fields so Stripe integration (Week 13) can be added later without major refactoring

---

Data Model Overview: How It All Fits Together

```
User Signs Up
    ↓
Creates Firebase Auth account
    ↓
Creates 'user' platform profile (auth domain)
    ↓
Discovers Salons on Marketplace (Week 2)
    ↓
Subscribes to Salon A (or purchases trial) → tenantUsers[salonA_userId] created
    ↓
    ├─ tenantUsers gets subscription metadata: tier=free_trial, status=trialing,
trialEndsAt=+14 days
    ├─ userTenantAccess[userId_salonA] created: subscriptionStatus=trialing,
unreadMessageCount=0
    └─ (Later: subscriptions[stripe_sub_123] created when payment processed - Week
13+)
    ↓
User Subscribes to Salon B → repeat above
    ↓
User Logs In
    ↓
Dashboard Query: SELECT * FROM userTenantAccess WHERE userId = 'uid_123'
    ↓
Shows Salon A + Salon B cards with badges, next appointments, loyalty points
    ↓
User Taps "Salon A" → Context switches, sets tenantId, navigates into protected
app
    ↓
User sees Salon A's data only (bookings, messages, loyalty, etc.)
    ↓
At 14 days (when trial ends):
    ├─ If no payment: automatic feature gating (booking disabled, messaging read-
only)
    ├─ If payment failed: grace period (N days), then suspended
    └─ If payment succeeded: tier=professional, status=active, nextBillingDate=in
1 month
```

Implementation Phases vs. Timeline

Phase 1: Weeks 1-2 (Foundation - DATA & SCHEMA)

- Build core multi-tenant architecture
- Extend tenantUsers with subscription fields

- Create userTenantAccess index
- No Stripe, no feature gating yet — just schema foundations

Why:

- Data layer must support multi-subscription users from day one
- Avoids major refactoring later to retrofit multi-tenant user support

Phase 2: Weeks 3-5 (Core Features - SINGLE CONTEXT)

- Build services, staff, scheduling, booking, messaging
- Assume all features work in a single tenant context
- Treat subscription as a "label in the database" for now

Why:

- Core features should be rock-solid before adding multi-context complexity
- Subscription becomes enforcement point later, not a design blocker now

Phase 3: Week 5.5 (Multi-Salon UX - CONTEXT SWITCHING)

- Build home dashboard showing all subscribed salons
- Build badge aggregation (unread message counts)
- Build context switcher + deep linking
- Still no Stripe, no feature gating — just UX for multi-context

Why:

- Users can now navigate between multiple salons
- All the plumbing (userTenantAccess, unread counts) is tested and reliable

Phase 4: Weeks 6-12 (Remaining Core Features)

- All other features (loyalty, campaigns, reviews, analytics, reports)
- Pilot launch with single Zara tenant
- All features work, no payment enforcement

Why:

- Stable pilot proves core business logic works
- Pilot can help validate UX/workflows before adding payment complexity

Phase 5: Weeks 13-20 (EXPANSION - Monetization, Onboarding, Marketplace, AI)

- Integrate Stripe API
- Implement payment success/failure workflows
- Add feature gating based on subscription tier
- Add suspension/grace period logic
- Upgrade upsell flows
- Implement salon onboarding wizard and client onboarding integration
- Launch marketplace features and analytics

- Launch AI assistant and prediction features

Why:

- At this point, everything is proven and stable
- Payment layer doesn't affect core feature logic
- Can be added independently without risking pilot

Key Architectural Decisions Locked In

1. Single Zarkili Account, Multiple Subscriptions

Pro:

- Users have one identity across all salons
- Shared profile, settings, loyalty (if desired)
- Simple auth/session model

Con:

- Adds userTenantAccess index layer for performance
- Unread count aggregation needed on every dashboard refresh

Alternative Rejected:

- Separate account per salon (simpler DB but worse UX, users forget passwords)

2. Denormalized userTenantAccess Index

Why:

- Query "list all salons for user" + unread counts is on critical path for dashboard
- Firebase query: `userTenantAccess WHERE userId = 'uid_123'` is fast and bounded
- Alternative: Query tenantUsers from each tenant (slower, network n-queries)

Sync Risk:

- userTenantAccess must stay in sync with tenantUsers during all updates
- Tests and docs include explicit sync points

3. Subscription Fields in tenantUsers (not separate collection)

Why:

- tenantUsers is the user-per-tenant record (already exists)
- Subscription metadata is per-user-per-tenant (same scope)
- Keeps related data together
- Simpler Firestore rules

4. Stripe Integration Post-Pilot

Why:

- Pilot doesn't need payment (Zara is already your partner)
- Core features must be stable before payment layer added
- Payment logic is independent of feature logic (can be added as a gating layer later)
- Reduces risk of pilot launch

What's NOT Changing (Preserved)

- ✅ Weeks 1-12 core delivery order remains intact
- ✅ Week 1-4 public/protected routing (`Landing → Login → App`)
- ✅ Core domain separation (auth, tenants, locations, staff, services, bookings)
- ✅ Security model (tenant isolation via Firestore rules)
- ✅ Single-context feature development (one tenant at a time when inside the app)
- ✅ No payment required for pilot launch

Next Steps

1. **Review & Validate** these documentation updates (Week 2 tasks are now more complex)
2. **Adjust Week 2 task estimates** if needed — adding userTenantAccess tests adds complexity
3. **Schedule Week 5.5 Planning** once Week 5 is mostly complete
4. **Execute the expanded Weeks 13-20 plan** after core pilot hardening

When You're Ready to Code:

1. Start with Week 1 tasks (unchanged)
2. Run **Global Guardrails Prompt** from updated prompts doc
3. When you reach **Week 2 Task 2.2**, use the updated prompt that includes subscription fields
4. Run **new Task 2.2.5** right after for userTenantAccess
5. Continue with Weeks 3-5 as planned

Appendix: Quick Reference

Subscription Tier Names (Recommended for Future Use)

- **Free Trial** (configurable, 14-day default)
- **Starter** (basic bookings, messages, limited reports)
- **Professional** (full features, multi-staff, campaigns)
- **Enterprise** (all + custom support, advanced analytics)

Subscription Statuses State Machine

```
free_trial → (at 14 days) → [payment attempt]
              ↓
           active ←————————— past_due (retry queue)
              ↓                    ↓
           cancelled         suspended (after grace period)
```

userTenantAccess Primary Use Case

```
Dashboard Query (critical path, must be fast):
db.collection('userTenantAccess')
  .where('userId', '==', currentUser.uid)
  .where('subscriptionStatus', '==', 'active')
  .get()

Result: [{tenantId, tenantName, logo, unreadCount, nextAppointment,
loyaltyPoints}, ...]
```

Questions for You Before Development Starts

1. **Subscription Tier Feature Matrix**: Which features are in which tier?
    - Example: Does Starter get messaging? Do all get reporting?
    - (Spreadsheet or doc can be created later, not a blocker)
2. **Free Trial Scope**: Is free trial only for new tenant signup, or can existing customer reorder trial?
3. **Payment Grace Period**: How many days before past_due becomes suspended?
    - Recommended: 7-10 days
4. **Annual Discount**: What % off for annual billing? (affects pricing in subscriptions collection)
5. **Marketplace Discovery**: When should users be able to discover/subscribe to new salons?
    - Now: Only during onboarding
    - Later: Anytime from dashboard
    - Recommended: Anytime (can always unsubscribe)

Status: ✅ Planning complete and documented. Ready for development kickoff.